# High Performance Computing – MPI Exercise


# Orazgali Azamat, BDA-2402


The programs were compiled with g++ -O2 -std=c++11 using the Microsoft MPI (MS-MPI) headers and library, and started with mpiexec -n <processes> program.exe (on Windows mpiexec is used instead of mpirun). Only the MPI header mpi.h and bits/stdc++.h are included.

Moreover, in this assignment I have to wrap all C++ codes with .bar files since the compiler of Dev-C++ (TDM-GCC 4.9.2) is too old for MS-MPI. The MPI programs compile in Dev-C++, but they crash with a segmentation fault inside MPI_Init and print nothing, even when started with mpiexec. The same code works with a newer g++ (MinGW-w64 16.1.0), so this compiler is called directly from the command line together with the MS-MPI mpiexec. Typing the long compile and mpiexec commands each time is inconvenient, so every program has its own .bat A bat file compiles its program and starts it with the required number of processes. build_run.bat runs all of them one after another. The source code itself is the same as required by the task and does not depend on the bat files.

## Part 1: MPI Processes and Ranks

Source code: hello_mpi.cpp. Output with 4 processes (one run):

Hello from process 2 of 4
Hello from process 3 of 4
Hello from process 0 of 4
Hello from process 1 of 4

| Number of Processes | Rank Values |
|---|---|
| 1 | 0 |
| 2 | 0, 1 |
| 4 | 0, 1, 2, 3 |

**1. What does MPI_Comm_size() return?**
The total number of processes in the communicator (for MPI_COMM_WORLD, the number of processes started with mpiexec).

**2. What does MPI_Comm_rank() return?**
The rank (identification number) of the calling process inside the communicator.

**3. If eight MPI processes are started, what are their possible rank values?**
0, 1, 2, 3, 4, 5, 6 and 7.

**4. What is MPI_COMM_WORLD?**
The default communicator that contains all the processes started for the program.

**5. Does every MPI process execute the same program?**
Yes. All processes run the same program (the same executable). They behave differently only because they have different ranks.

**6. Does the order in which the processes print their messages always remain the same?**
No. The processes run independently, so the order of the messages is not guaranteed and can change between runs.

## Part 2: Different Work for Different Processes

Source code: rank_square.cpp. Each process calculates x = (rank + 1) * (rank + 1). Output with 4 processes:

Process 3: x = 16
Process 1: x = 4
Process 2: x = 9
Process 0: x = 1

**1. Does each process have a different value of rank?**
Yes. Every process has its own unique rank.

**2. Does every process have its own copy of variable x?**
Yes. MPI processes have separate memory, so each one has its own x.

**3. Does process 0 directly access the variable x belonging to process 1?**
No. A process cannot read the memory of another process directly. Data has to be sent with messages.

**4. Is any communication required for these calculations?**
No. Each process uses only its own rank, so no communication is needed.

## Part 3: Point-to-Point Communication

Source code: send_receive.cpp. Process 0 sends the value 100 to process 1 with MPI_Send, process 1 receives it with MPI_Recv, multiplies it by 2 and prints it. Output with 2 processes:

Result = 200

**1. Which process is the sender?**
Process 0.

**2. Which process is the receiver?**
Process 1.

**3. Why does the receiver need to know the source process?**
MPI_Recv has to know which process the message comes from so that it takes the message from the right sender (or it can use MPI_ANY_SOURCE if the sender is not important). It is also needed to match the message correctly when several processes send data.

**4. What is the purpose of the message tag?**
The tag is an integer label of the message. It lets the receiver distinguish different messages from the same sender and match each receive with the right send.

**5. Is this an example of point-to-point or collective communication?**
Point-to-point communication, since it is between exactly two processes.

## Part 4: Broadcast

Source code: broadcast.cpp. Process 0 sets N = 10000000 and calls MPI_Bcast with process 0 as the root. Output with 4 processes:

Process 1: N = 10000000
Process 0: N = 10000000
Process 2: N = 10000000
Process 3: N = 10000000

**1. Which process initially knows the value of N?**
Process 0 (the root).

**2. Which processes know N after the broadcast?**
All processes in the communicator.

**3. Does MPI_Bcast() need to be called only by the root, or by all processes?**
By all processes. The root sends and the others receive using the same call.

**4. Why is repeatedly calling MPI_Send() from the root less convenient than using a broadcast?**
The root would need a separate MPI_Send for each process and each of them needs a matching MPI_Recv, so the code is longer and depends on the number of processes. MPI_Bcast is one call and the library can also distribute the data more efficiently.

## Part 5: Parallel Sum

Source code: parallel_sum.cpp (this file also contains Part 6). N = 10,000,000 is set on process 0 and broadcast. Each process calculates its own range and sums it with a loop. Output with 4 processes:

Process 0: start = 1, end = 2500000, local sum = 3125001250000
Process 1: start = 2500001, end = 5000000, local sum = 9375001250000
Process 2: start = 5000001, end = 7500000, local sum = 15625001250000
Process 3: start = 7500001, end = 10000000, local sum = 21875001250000

## Part 6: Reduction

The local sums are combined with MPI_Reduce (MPI_SUM, MPI_LONG_LONG, root 0). Only process 0 prints:

Total sum = 50000005000000

Check with the formula N(N+1)/2 = 10000000 * 10000001 / 2 = 50000005000000. The results are equal.

**1. Why can each process calculate its local sum independently?**
The ranges of numbers are different and do not overlap, and the sum of one range does not need any data from another range.

**2. What information needs to be communicated after the local calculations are finished?**
The local sums of all processes must be sent to process 0.

**3. What operation does MPI_Reduce() perform in this program?**
It adds (MPI_SUM) the local sums of all processes into one total sum.

**4. Which process receives the final result?**
Process 0, because it is the root.

**5. What would be different if MPI_Allreduce() were used instead?**
The final sum would be delivered to every process, not only to the root, and there would be no root argument.

## Part 7: Scatter and Gather

Source code: scatter_gather.cpp. Process 0 holds A = {10, 20, 30, 40}, scatters one element to each of the 4 processes, every process multiplies its value by rank + 1, and the results are gathered back on process 0. Output:

Final array: 10 40 90 160

**1. What is the main purpose of MPI_Scatter()?**
To split an array stored on the root process into equal parts and send one part to each process.

**2. What is the main purpose of MPI_Gather()?**
To collect the pieces from all processes and put them into one array on the root process.

**3. Which process contains the complete input array before MPI_Scatter()?**
Process 0.

**4. Which process contains the complete output array after MPI_Gather()?**
Process 0.

## Part 8: Measuring Execution Time

Source code: parallel_sum_timing.cpp. It is the program from Parts 5 and 6 with MPI_Wtime() added, and N was increased to 1,000,000,000 so that the time can be measured. The time covers the local calculation and the reduction. Three runs were made for each number of processes; the table shows the average.

| MPI Processes | Execution Time (seconds) | Speedup |
|---|---|---|
| 1 | 0.2489 | 1.00 |
| 2 | 0.1377 | 1.81 |
| 4 | 0.0761 | 3.27 |

**1. Does increasing the number of MPI processes always reduce execution time?**
No, not always. Here it did, but when the problem is small or the communication is expensive, more processes can make it slower.

**2. Is the speedup equal to the number of processes?**
No. It was 1.81 with 2 processes and 3.27 with 4 processes, which is less than the ideal 2 and 4.

**3. What additional overhead exists in an MPI program that does not exist in the serial version?**
Starting the processes, MPI_Init and MPI_Finalize, the broadcast of N, the reduction of the results, and the waiting of processes for each other.

**4. Why can communication reduce the benefit of parallel computation?**
Time spent sending and waiting for messages is not useful calculation. If the computation per process is small compared to the communication time, the speedup becomes small.

## Part 9: Short Discussion

**1. What is the purpose of MPI_Init()?**
It initializes the MPI environment. It must be called before any other MPI function.

**2. What is the purpose of MPI_Finalize()?**
It shuts down the MPI environment. No MPI functions can be used after it.

**3. What is an MPI rank?**
The unique number of a process inside a communicator, from 0 to size - 1.

**4. What is an MPI communicator?**
A group of processes that can communicate with each other. MPI_COMM_WORLD is the communicator with all processes.

**5. What is the difference between point-to-point and collective communication?**
Point-to-point involves two processes (one sender and one receiver, for example MPI_Send and MPI_Recv). Collective communication involves all processes of a communicator (for example MPI_Bcast, MPI_Reduce).

**6. What is the difference between MPI_Send() and MPI_Bcast()?**
MPI_Send sends a message from one process to one specific process and needs a matching MPI_Recv. MPI_Bcast sends the data from the root to all processes, and all processes call it.

**7. What is the difference between MPI_Reduce() and MPI_Gather()?**
MPI_Gather only collects the values from all processes into an array on the root. MPI_Reduce collects the values and also combines them with an operation (for example sum) into one result.

**8. Why does MPI work well for distributed-memory systems?**
Every process has its own memory and data is exchanged only with explicit messages, which matches computers (nodes) that do not share memory and are connected by a network.

**9. What is one important difference between MPI processes and OpenMP threads?**
MPI processes have separate memory and communicate by messages, while OpenMP threads share the same memory of one process.